Sentinel

Unified Viewing & Metadata Analytics — Model 2

Technical Proposal — High-Level Design (HLD)
Gujarat Police Innovation Challenge 2026

Sentinel — High-Level Design

Model 2: Unified Viewing & Metadata Analytics — Gujarat Police Innovation Challenge 2026

1. Overview and model choice

We implement **Model 2** exactly as specified: a unified viewing platform that connects directly to each camera/VMS system via RTSP, without introducing a middleware or federation layer. Departmental VMS/storage systems are untouched — we consume live streams only, we never push to or control the gateway.

Why Model 2, not 3/4: given the challenge's real timeline and the diversity of 26 departments' existing VMS/storage (some cloud, some local; 7–15+ day retention; different vendors), a federation/central-VMS layer (Model 3/4) is a much larger, multi-department integration program — appropriate for a phased statewide rollout, not a hackathon prototype. Model 2 delivers the two things the evaluation actually tests — unified live viewing and AI-powered cross-camera vehicle tracing — without requiring any department to change how it stores or manages its own footage. It is also the lowest-risk entry point for the eventual statewide expansion described in section 7: departments can be onboarded into central viewing/analytics one at a time, on existing infrastructure, before any heavier federation investment is made.

2. Architecture

```
Departmental CCTV / VMS (RTSP, ONVIF, vendor APIs)
        |  direct connection, credentials embedded per-camera, no relay/storage copy
        ▼
Resilient Capture Layer  (capture/)
  - one CameraCapture per active camera: TCP-forced RTSP, PTS-derived timestamps,
    exponential-backoff reconnect, decoder-warning tolerance, scene-cut recovery
  - camera catalogue pulled live from the gateway's own directory endpoint —
    no hardcoded camera IDs/URLs anywhere in the system
        ▼
ANPR Analytics Pipeline  (analytics/)
  - configurable frame sampling (every Nth frame) for CPU-only feasibility
  - YOLO-class plate detector (pretrained, CPU inference)
  - PaddleOCR on cropped plate regions, with Indian-plate OCR-confusion normalization
        ▼
Watchlist Correlation + Alerting  (watchlist/)
  - SQLite watchlist (plate, reason/category, date added)
  - every OCR read is normalized and fuzzy-matched (confusion-aware) against the
    watchlist in real time
  - a match writes an alert row: camera, plate, PTS-derived timestamp, confidence,
    matched entry
        ▼
Route Reconstruction  (vehicle_trace/)
  - given a plate, returns every (camera, location, timestamp) detection in
    chronological order — this is the vehicle-tracing capability the live
    evaluation exercises directly
        ▼
Unified Control-Room View  (dashboard/)
  - live alert feed, camera grid with last-seen plate per camera, plate search /
    route view, watchlist administration — auto-refreshing, no manual reload
```

No frame is ever written to persistent storage as a side effect of this pipeline — only structured metadata (plate text, camera, timestamp, confidence) is retained. This keeps storage/bandwidth costs low and sidesteps most video-retention/privacy concerns; full video remains wherever the owning department already keeps it, for whatever retention period they already use.

3. Live stream ingestion

- Every capture explicitly forces RTSP over TCP. UDP is accepted by many gateways but produces silently corrupted frames across NAT/firewalls, which manifests as mysterious model errors rather than an obvious connection failure — forcing TCP eliminates that failure class entirely.
- Timing (frame intervals, dwell time, "last seen") is derived exclusively from each frame's presentation timestamp (PTS), never from wall-clock arrival time or a camera's self-reported frame rate. Both of those are unreliable on real gateways: a freshly-connected client is handed a buffered group-of-pictures that can arrive faster than real time, and reported FPS routinely doesn't match actual delivery.
- Reconnection uses exponential backoff (starting ~2s, capped ~30s) — feeds are expected to restart periodically as a normal operating condition, not a fatal error, and a tight reconnect loop would itself degrade the gateway.
- Decoder warnings at the moment of connecting (missing reference frames, general warm-up noise before the first keyframe) are logged, not treated as fatal — real testing against a live 30-camera grid confirmed this is a routine, self-correcting condition on both H.264 and H.265 streams, not an error condition.
- Camera inventory (IDs, display names, per-camera stream URLs) is always read live from the gateway's own camera-directory endpoint, cached locally for resilience, and never hardcoded — new cameras or a changed camera set require no code change.
- Only cameras actively being processed are held open; a capture is closed as soon as it is no longer needed. At statewide scale this is what keeps the platform's own connection footprint on each department's gateway proportional to actual demand rather than the full camera count.

4. Watchlist integration and alerting workflow

The watchlist is a simple, swappable schema: plate number, reason/category, date added. For this prototype we populate our own representative watchlist (explicitly permitted by the challenge rules); in a production deployment the same schema would be fed from the departments' existing systems — **VAHAN** (vehicle registration) and **eGujCop/CCTNS** (stolen-vehicle and wanted-persons records) are the natural sources for vehicle-side entries, reachable as a scheduled sync or an event feed rather than a tight synchronous dependency, so a watchlist-source outage never blocks live alerting.

Every OCR read — not just ones a human reviews — is normalized (uppercase, punctuation stripped) and checked against the watchlist immediately, in two passes: an exact/near match against common OCR confusions (0/O, 1/I, 5/S, 8/B, 2/Z, which we specifically handle for Indian plate formats), then a fuzzy-similarity fallback for anything else. This two-pass design keeps precision high on the common case while still catching single-character OCR errors, which are frequent on real CCTV footage at night or at distance.

A match immediately writes an alert record — camera ID, plate, PTS-derived timestamp, match confidence, and the matched watchlist entry's reason/category — and surfaces in the dashboard's live alert feed without any manual refresh. Alert prioritization (by category — stolen/wanted/suspect/missing) and role-based routing to the appropriate desk are natural next steps once integrated with a real command-center notification channel (see section 7).

5. AI/analytics approach

ANPR is the mandatory analytic and the only one implemented in this prototype, deliberately — face recognition and other analytics are explicitly out of scope given the timeline, and adding them without doing ANPR well would not have served the evaluation.

- **Detection:** a pretrained, CPU-feasible YOLO-nano-class plate detector (not trained from scratch, given the timeline — see the presentation for the licensing note on the specific pretrained weights used).
- **OCR:** PaddleOCR (PP-OCRv6) on the cropped plate region, upscaled before recognition — see the methodology note below for why this replaced our initial choice of EasyOCR, and with what evidence.
- **Sample rate:** every Nth frame is processed (configurable), not every frame — necessary for real-time CPU-only throughput across multiple concurrent camera feeds on commodity hardware, and sufficient because a vehicle is visible across many consecutive frames as it crosses a camera's field of view.
- **Tiled full-frame detection** (re-running the detector per-region at native resolution instead of one downscaled whole-frame pass) is available as a configurable, per-camera option for cameras where plates are consistently too small for a single pass to find — at the cost of proportionally more CPU per sampled frame.

Methodology note: how we found and fixed our own plate-legibility gap

This is worth documenting in detail because it is exactly the kind of practical, evidence-driven iteration the evaluation is looking for, not just a design choice made on paper.

Finding 1 — camera angle, not just camera count, determines ANPR viability. Initial testing sampled cameras more or less in catalogue order and found mostly wide-angle, overhead traffic-junction cameras (optimized for scene coverage and incident monitoring, not close-range plate capture) — plates on distant vehicles were frequently not resolvable at any zoom, which is a real, industry-recognized limitation of general-purpose traffic CCTV, not a pipeline defect. Rather than accept that as a grid-wide verdict, we went back to the camera catalogue's own naming (`cameras.json` gives a human display name per camera) and looked for signal in it: one entry named "Tri Mandir Adalaj Tollnaka" ("Tollnaka" = toll plaza in Gujarati) stood out as a plausibly close-range, purpose-different camera class. It was — testing it directly gave us the first genuinely legible plate crop we had seen, a truck stopped at the toll barrier, front-on, no obstruction. **Lesson for the statewide design:** camera metadata/naming conventions are a cheap, high-value signal for triaging which cameras in a large heterogeneous grid are ANPR-viable versus purely for situational viewing — worth standardizing on onboarding (see §8).

Finding 2 — OCR engine choice, evidenced not assumed. On that same legible crop, our initial OCR engine (EasyOCR) never exceeded ~0.2–0.4 confidence across extensive preprocessing tuning (multiple upscale factors, unsharp masking, detection-threshold tuning). We swapped to PaddleOCR (PP-OCRv6) on the identical crop, no other changes, and got a full read at **0.87 confidence**. (One infrastructure snag along the way: PaddleOCR's default MKL-DNN CPU acceleration crashed outright on this host with a native oneDNN/PIR runtime error — disabling it, `enable_mkldnn=False`, fixed that; pure CPU inference without it is somewhat slower but correct, which is the right tradeoff at our current frame-sampling rate.)

Finding 3 — the real remaining bug was in our own bounding-box crop, not the OCR engine. Even after the OCR swap, running the *actual* pipeline (detector's own output box → crop → OCR) still failed, while our hand-cropped test image succeeded. The detector's tight bounding box was clipping characters at the edge — enough to break OCR outright even though the plate was otherwise legible. We swept padding fractions against the real detector output (not a synthetic test) and found padding the box by 75% of its own width/height is a genuine sweet spot: less padding still clips characters, and (counterintuitively) more padding (100%+) pulls in enough surrounding clutter to confuse OCR again. This is now the default in `PlateBox.crop()`.

Result, verified through the actual pipeline classes (not a standalone script) on live sandbox footage: detector confidence 0.42, OCR confidence 0.79–0.96 depending on the vehicle, full plausible Indian plates read correctly (e.g. `GJ05AU9828`, a valid Gujarat-format plate, read at 0.96 OCR confidence; the same physical vehicle's plate read as `BV2807` / `8V2807` / `EV2807` across five independent detections within 16 seconds — a useful cross-check that the pipeline isn't just getting lucky once, and see below for what that variation itself surfaced). This isn't limited to one camera: running the pipeline simultaneously against three independently-confirmed ANPR-viable cameras (`cam12` the toll plaza, `cam06` a gate camera, `cam22` a bypass-road camera) over a multi-hour unattended run produced 139+ detections clearing the OCR confidence threshold across all three — including full plausible plates on each (e.g. `GJ2832AGOC` at 0.95 confidence on `cam06`) — the large majority fully plausible plate strings rather than fragments, once the fixes above were in place.

Finding 4 — the same real-vehicle data surfaced two further matching bugs, both fixed. The `BV2807` / `8V2807` / `EV2807` variation above wasn't just a curiosity — running it through our own watchlist-matching and route-reconstruction code broke both of them, in different ways: - `watchlist/matcher.py`'s OCR-confusion-variant generator used `str.replace()`, which substitutes *every* occurrence of a character in a string. On `8V2807` (which contains two `8`s — one a misread `B`, one a genuine digit), this corrupted the real digit too and produced `BV2B07`, not the intended `BV2807` — so a genuine watchlist hit would have been silently missed. Fixed to swap one character position at a time. - `vehicle_trace/route_reconstruction.py`'s plate search matched the *exact* normalized string only, so searching for `BV2807` returned only 2 of the vehicle's 4 confusion-linked detections, under-reporting its route. Fixed to search across the same confusion-variant set used by watchlist matching.

Full detail, before/after evidence, and the honest boundary of what confusion-variant matching can and can't catch (it does not treat `E` ↔ `B` as a confusion, since they aren't visually similar characters — that miss is deliberate, not an oversight) is in `docs/evidence/trace_demonstration.md`.

Production implication: we recommend pairing wide-area situational cameras with a smaller number of dedicated, close-range ANPR gantry/bollard cameras at chokepoints (toll plazas, checkpoints, junction approach lanes) for reliable plate capture — this platform's unified viewer works identically over both camera types, and the ANPR pipeline automatically benefits wherever close-range coverage already exists, exactly as demonstrated here.

6. Alert notification workflow

Current prototype: alerts land in the SQLite alert log and the dashboard's live feed in real time (sub-second from detection to visible alert). For an operational deployment, the same alert-write hook (`watchlist/alerting.py`) is the integration point for: - push notification to a command-center console / mobile app, - SMS/email escalation for high-priority categories (stolen, wanted), - an audit trail of who viewed/actioned each alert (role-based access, see below).

7. Scalability, interoperability, security, performance — path to ~80,000 cameras

Scalability. The capture layer already treats "which cameras are open" as a runtime-configurable set pulled from a live directory, not a static list — the natural scale-out path is horizontal: shard the camera set across multiple capture+ANPR worker processes/nodes (e.g. by department or district), each reporting detections into a shared watchlist/alert store, with the dashboard querying across shards. Because each `CameraCapture` and `AnprWorker` is independent and only holds resources for cameras it's actively processing, this scales close to linearly with added compute — no component in the current design assumes a single-process, single-machine camera count.

- **Central vs. edge compute:** central CPU inference (as built) is appropriate for a pilot/regional scale; at 80,000 cameras, edge or regional pre-filtering (motion/ROI gating before a frame is even sent for full inference, or edge NVR-side inference where hardware supports it) becomes necessary to keep central compute and backhaul bandwidth bounded. The frame-sampling and confidence-threshold parameters in this prototype are exactly the knobs that would move to edge-side config in that model.
- **GPU/accelerator requirements:** this prototype runs CPU-only by design constraint and stays real-time on a 40-core/64GB node for a handful of concurrent cameras. At city/state scale, GPU or NPU acceleration (or edge accelerators like Jetson-class devices at high-priority chokepoints) is the realistic path to keeping per-camera inference cost low enough for tens of thousands of feeds; the detector/OCR abstraction (`analytics/plate_detector.py` , `analytics/ocr.py`) is swappable without touching the rest of the pipeline.
- **Bandwidth:** only RTSP streams for cameras actively being processed are pulled by this platform — we never mirror the full 80,000-camera grid continuously. Low- bandwidth sites should be prioritized for edge pre-filtering (send metadata, not full video, upstream) rather than raw stream relay.
- **Storage:** this design intentionally does not centrally store video — only structured metadata (detections, alerts). Hot storage is the SQLite/production-DB detection and alert log (small, queryable, cheap to keep long-term); any full-video retention decision stays with each department's existing hot/warm/cold policy (7–15+ days as already documented per department) — this platform does not change that responsibility.
- **Interoperability:** the capture layer's only hard requirement of a camera is an RTSP (or ONVIF/vendor-SDK, as an alternative capture adapter) endpoint and TCP transport — this covers the large majority of modern IP CCTV/VMS without touching departmental infrastructure. A department whose system exposes only a vendor SDK would need a thin adapter that produces the same `get_frame() -> (frame, pts_ms, camera_id)` interface the rest of the pipeline already consumes — no other component needs to change.
- **Security:** credentials are per-camera/per-gateway and never logged in plaintext (verified in this codebase — RTSP/WHEP URLs are redacted before any log line). Production hardening beyond this prototype: TLS for all control-plane traffic, short-lived per-session credentials rather than long-lived embedded ones, RBAC on the dashboard (who can view which department's cameras, who can action an alert, full audit logging of alert views/actions), and network segmentation so the analytics platform is a consumer on each department's network, never a control path back into their VMS (we already never call any gateway control API — consume-only by design).
- **Monitoring/health:** the capture layer already exposes per-camera connection state, frame counts, and reconnect counts (`CaptureManager.status()`); at scale this feeds directly into a health dashboard / alerting system for camera-down detection — itself a useful operational signal independent of ANPR.
- **High availability / disaster recovery:** each capture/ANPR worker is independent and stateless beyond its own reconnect backoff, so a worker or node failure affects only the cameras it owned — restart is a cold reconnect using the same live camera catalogue, with no special recovery procedure. The watchlist/alert store is the one piece of durable state and should be replicated/backed up using standard database HA practice at production scale (this prototype's SQLite is a pilot-scale choice; swapping to PostgreSQL is a schema-compatible change, not a redesign).

8. What we need from participating departments

- A reachable RTSP (or ONVIF/vendor-SDK) endpoint per camera intended for onboarding, and confirmation of codec/resolution/frame-rate characteristics per camera or camera group (already handled generically in this build, but

departments should expect to be asked).

- Network reachability from the analytics platform to each camera's stream endpoint (firewall allowance for the relevant ports), scoped read-only.
- For watchlist correlation to be meaningful beyond a demo dataset: an integration contact/API access pattern for VAHAN and eGujCop/CCTNS data (even a scheduled export is sufficient to start).
- Departments' existing retention policy per camera group, so hot/warm/cold storage planning for the *metadata* layer (this platform) can be sized correctly relative to each department's own video retention (which this platform does not change).